

codehive

Python Exception Handling

Mastering Try, Except, Else, and Finally

PYTHON CORE COMPETENCY SERIES

1. Introduction to Exceptions

When an error occurs, Python will normally stop and generate an error message. These errors are called **Exceptions**.

Instead of letting the program crash, we use the **try...except** block to handle these situations gracefully.

- **try:** Tests a block of code for errors.
- **except:** Handles the error identified.

2. Basic Try...Except

Without a try block, a variable that hasn't been defined will crash the entire program.

Example: Handling a NameError

```
try:
    print(x) # x is not defined
except:
    print("An exception occurred")
```

In this scenario, because 'x' does not exist, the program jumps straight to the except block instead of stopping production.

3. Handling Multiple Exceptions

You can define specific blocks of code for specific kinds of errors. This allows for more precise debugging and user feedback.

```
try:
    print(x)
except NameError:
    print("Variable x is not defined")
except TypeError:
    print("Variable is the wrong type")
except:
    print("Something else went wrong")
```

Python matches the error to the specific exception named. If none match, it falls back to the generic except block.

4. The 'Else' Keyword

The **else** block lets you execute code only when no errors were raised in the try block.

Example Usage:

```
try:
    print("Hello Codehive")
except:
    print("Something went wrong")
else:
    print("The operation was successful!")
```

This is useful for logic that should only proceed if the "risky" code in the try block worked perfectly.

5. The 'Finally' Keyword

The **finally** block is the most resilient block in Python. It is executed **regardless** of whether an error occurred or not.

This block is primarily used for "cleanup" actions, such as closing database connections or files.

```
try:
    print(x)
except:
    print("Something went wrong")
finally:
    print("The 'try except' process is complete.")
```

6. Practical Application: Files

One of the best uses of `finally` is closing file objects so that they don't consume system resources if an error occurs during writing.

```
try:
    f = open("data.txt")
    try:
        f.write("Lorum Ipsum")
    except:
        print("Error writing to file")
    finally:
        f.close()
        print("File connection closed safely")
except:
    print("Error opening the file")
```

7. Raising Exceptions Manually

As a developer, you can choose to "throw" or **raise** an exception if a certain business condition is met.

Use the `raise` keyword to stop the program with a custom message.

```
x = -5
if x < 0:
    raise Exception("Sorry, no numbers below zero")
```

The program will halt and display your custom error string to the user.

8. Raising Specific Types

You can define what kind of error to raise (e.g., `TypeError`, `ValueError`, `IndexError`).

Example: Restricting Input Types

```
name = 123
if not type(name) is str:
    raise TypeError("Only strings are allowed for names")
```

This ensures that your functions receive the data types they expect before the logic proceeds.

9. Common Built-in Exceptions

Familiarize yourself with these common error types encountered in Python:

Exception	Cause
ArithmeticError	Numeric calculation errors
ImportError	Imported module not found
IndexError	Sequence index out of range
KeyError	Dictionary key not found
ValueError	Function gets right type but wrong value

10. Summary & Knowledge Check

Exercise

In a try...except block, there is a certain block that if specified, will be executed **regardless** if the try block raises an error or not. What is the name of this block?

Correct Answer: `finally`

You have successfully completed the Codehive guide to Python Exception Handling.